

Steps and Basis for Edge Connections:

1. Priority Queue Construction:

- The FFT output (`fftOutput`) is stored in a **priority queue** (using `priority_queue_from_array` and `priority_queue_build_max_heap`), and then the heap is printed (`priority_queue_print_heap`). This step organizes the FFT magnitudes in a heap structure based on their values (peaks and valleys).

2. Graph Creation:

- A **graph** is created with `create_graph(&graph, FFT_LENGTH)`, which initializes the graph structure with a size based on the FFT length. Each vertex in the graph represents a **magnitude** from the FFT results.

3. Vertices Addition:

- For each value in the heap (`pq.heap`), a **vertex** is added to the graph. Each vertex corresponds to an FFT output magnitude (likely representing voltage amplitude in the frequency domain).

4. Edge Creation Based on Peaks and Valleys:

- The core logic for adding edges checks for **peaks** and **valleys** in the magnitude values of the FFT results:
 - For each vertex in the graph, the code compares the **current value** (`curr_val`) with its **previous** (`prev_val`) and **next** (`next_val`) values.
 - **Peak detection:** If the current value is **greater than** both its previous and next values, it's considered a **peak**. Edges are added between the peak and its neighboring **valleys** (i.e., vertices before and after).
 - **Valley detection:** If the current value is **smaller than** both its previous and next values, it's considered a **valley**. Similarly, edges are added between the valley and its neighboring **peaks**.

5. Graph Visualization:

- Finally, the graph is printed (`print_graph(&graph)`), showing the vertices (peaks and valleys) and their connections (edges).

Edge Connection Summary:

- **Peaks** are connected to adjacent **valleys** and vice versa.
- The edges are added based on the **relative magnitude** comparisons of adjacent FFT results (magnitude values), creating a **graph** where peaks and valleys are linked.